

Robot Inteligente con Control de Voz, IA y Obstáculos

Documento Explicativo del Código (Versión con obstáculos)

David Santiago Piña Jaramillo
Asignatura: Inteligencia Artificial y Minirobots

June 3, 2026

Contents

1	Introducción	3
2	Visión general del sistema	3
3	Fundamentos de Inteligencia Artificial utilizados	3
3.1	Reconocimiento de voz (Speech-to-Text)	4
3.2	Procesamiento de Lenguaje Natural (NLP)	4
3.3	Representación TF-IDF (Term Frequency – Inverse Document Frequency)	4
3.4	Clasificación mediante aprendizaje supervisado	4
3.4.1	Fase de entrenamiento	5
3.4.2	Fase de inferencia	5
3.5	Diagrama de flujo del sistema	5
4	Explicación rigurosa bloque a bloque	6
4.1	Bloque 0: Importaciones y constantes globales (líneas 1–70)	6
4.2	Bloque 1: Función <code>obstaculos()</code> (líneas 15–46)	7
4.3	Bloque 2: Clase <code>IACommandClassifier</code> (líneas 73–152)	7
4.4	Bloque 3: Clase <code>Robot</code> con obstáculos (líneas 155–235)	7
4.5	Bloque 4: Enrutador de intenciones <code>execute_ia_intent</code> (líneas 238–276)	8
4.6	Bloque 5: Clase <code>VoiceController</code> (líneas 279–321)	8
4.7	Bloque 6: Funciones gráficas	8
4.7.1	<code>draw_robot(surf, robot)</code> (líneas 324–375)	8
4.7.2	<code>draw_obstacles(surf, robot)</code> (líneas 410–423)	9
4.7.3	<code>draw_panel(...)</code> (líneas 378–407)	9
4.8	Bloque 7: Bucle principal y código que corre todas las clases y definiciones <code>main()</code> (líneas 426–506)	9

5	Manual de usuario	9
5.1	Requisitos de instalación	9
5.2	Instalación de librerías (ejecutar en terminal)	10
5.3	Ejecución en VS Code	10
5.4	Controles y comandos	10
5.4.1	Comandos de voz (en español)	10
5.4.2	Controles de teclado	10
5.5	Interfaz gráfica	11
6	¿Por qué no funciona en Google Colab?	11
7	Posibles errores y soluciones	11
8	Limitaciones del sistema	12
9	Conclusión	12

1 Introducción

El presente documento describe en detalle el funcionamiento del programa `Robot_obs.py`, una versión un robot virtual que incorpora **obstáculos sólidos**. El robot puede ser controlado mediante comandos de voz (utilizando lenguaje natural y una pequeña red neuronal) o mediante el teclado. La interfaz gráfica se implementa con Pygame.

Los objetivos de este documento son:

- Explicar rigurosamente cada bloque del código, desde las importaciones hasta el bucle principal.
- Mostrar cómo se definen y gestionan los obstáculos.
- Proporcionar un manual de usuario completo (comandos de voz, teclado, elementos de la interfaz).
- Indicar los requisitos de instalación y ejecución (Python, librerías, entorno local vs. Google Colab).

El código completo se incluye en Github como `Robot_obs.py`.

2 Visión general del sistema

El programa consta de cinco módulos principales:

1. **Configuración del mundo:** define la cuadrícula, colores y direcciones.
2. **Clasificador de intenciones (IA):** utiliza TF-IDF y regresión logística para traducir frases en comandos.
3. **Controlador de voz en hilo:** captura audio y lo transcribe a texto.
4. **Interfaz gráfica:** dibuja la cuadrícula, obstáculos, robot y panel de estado.

La comunicación entre el hilo de voz y el hilo principal se realiza mediante una cola simple.

3 Fundamentos de Inteligencia Artificial utilizados

En esta sección se detallan los conceptos de inteligencia artificial que hacen posible que el robot interprete comandos de voz en lenguaje natural. El sistema combina **reconocimiento de voz**, **procesamiento de lenguaje natural (NLP)** y **aprendizaje supervisado**.

3.1 Reconocimiento de voz (Speech-to-Text)

El programa utiliza la librería `speech_recognition` que a su vez emplea la API de Google Speech. El proceso es el siguiente:

1. El usuario habla una frase (ej. *“avanza un poco”*).
2. El micrófono captura la señal de audio analógica y la convierte en digital.
3. La librería envía el audio a los servidores de Google mediante una conexión HTTPS.
4. Google devuelve una cadena de texto con la transcripción (ej. *“avanza un poco”*).
5. Este texto se entrega al clasificador de intenciones.

3.2 Procesamiento de Lenguaje Natural (NLP)

El robot debe entender que frases diferentes pueden tener la misma intención. Por ejemplo:

- *“camina”, “avanza”, “sigue derecho”* → **adelante**
- *“retrocede”, “reversa”, “un paso atrás”* → **atras**

Para ello se entrena con ejemplos, una biblioteca creada en el código.

3.3 Representación TF-IDF (Term Frequency – Inverse Document Frequency)

Antes de aplicar cualquier modelo matemático, las frases deben convertirse en vectores de números reales. El método elegido es **TF-IDF**. Dada una frase (documento) d y un término t , se calcula:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

donde:

- $\text{TF}(t, d) = \frac{\text{número de veces que aparece } t \text{ en } d}{\text{número total de términos en } d}$ (frecuencia relativa del término en la frase).
- $\text{IDF}(t) = \log \left(\frac{\text{número total de frases en el entrenamiento}}{\text{número de frases que contienen } t} \right)$ (inverso de la frecuencia anterior donde se da fuerza a palabras menos repetidas).

3.4 Clasificación mediante aprendizaje supervisado

El problema consiste en asignar a cada vector TF-IDF una etiqueta de intención.

3.4.1 Fase de entrenamiento

Se construye una tabla de ejemplos etiquetados manualmente:

Frase (entrada)	Etiqueta (salida deseada)
“avanza”	adelante
“camina”	adelante
“retrocede”	atras
“gira a la izquierda”	izquierda
“gira a la derecha y avanza”	mueve_derecha

El vectorizador TF-IDF transforma cada frase en un vector numérico. El algoritmo de regresión logística aprende una función que, dado un vector , devuelve la probabilidad de que pertenezca a cada clase. La clase con mayor probabilidad es la predicción.

3.4.2 Fase de inferencia

Cuando el usuario dice una frase nueva (ej. “*anda hacia adelante*”), el sistema:

1. La transcribe a texto.
2. La convierte a vector TF-IDF usando el mismo vectorizador (sin reentrenar).
3. El modelo entrenado calcula las probabilidades $P(\text{adelante}|\mathbf{x})$, $P(\text{atras}|\mathbf{x})$, etc.
4. Si la máxima probabilidad supera un umbral (0.40 o 0,35 dependiendo de que tan exacto queramos que sea el sistema), se acepta la intención; en caso contrario se rechaza.

3.5 Diagrama de flujo del sistema

El siguiente diagrama resume el proceso completo desde la voz hasta la acción del robot.

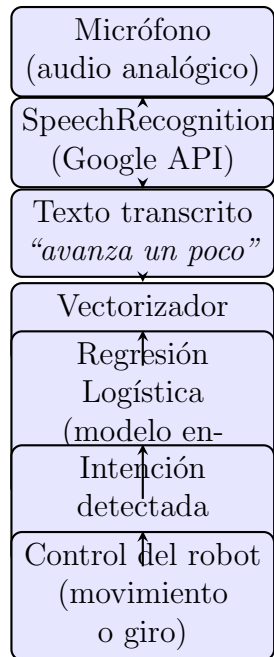


Figure 1: Flujo de procesamiento de la IA: desde la voz hasta la acción.

4 Explicación rigurosa bloque a bloque

4.1 Bloque 0: Importaciones y constantes globales (líneas 1–70)

```
1 import math
2 import sys
3 import threading
4 import pygame
5 import speech_recognition as sr
6 from sklearn.feature_extraction.text import TfidfVectorizer
7 from sklearn.linear_model import LogisticRegression
```

Listing 1: Importaciones y constantes

- `math`: funciones trigonométricas para dibujar la nave y operaciones matemáticas.
- `sys`, `threading`: control de salida y ejecución concurrente.
- `pygame`: gráficos y eventos.
- `speech_recognition`: acceso al micrófono y reconocimiento de voz (Google API).
- `sklearn`: herramientas de aprendizaje automático.

4.2 Bloque 1: Función `obstaculos()` (líneas 15–46)

```
1 def obstaculos():
2     obs = set()
3     for i in range(5, 10):
4         obs.add((i, 8))
5         obs.add((i, 12))
6     for j in range(6, 14):
7         obs.add((14, j))
8         obs.add((6, j))
9     obs.add((10, 10))
10    obs.add((3, 15))
11    obs.add((17, 4))
12    obs.add((4, 17))
13    return obs
```

Listing 2: Definición de obstáculos

¿Cómo coloca obstáculos? Crea un conjunto vacío y añade tuplas (x, y) mediante bucles `for`. Los rangos generan barras horizontales y verticales, y los puntos sueltos crean obstáculos aislados. Este conjunto se devuelve y será asignado a `robot.obstacles`. Cualquier coordenada presente en él es considerada bloqueada.

4.3 Bloque 2: Clase `IACommandClassifier` (líneas 73–152)

Propósito: clasificar frases de voz en una de siete intenciones: `adelante`, `atras`, `izquierda`, `derecha`, `mueve_izquierda`, `mueve_derecha`, `desconocido`.

- `__init__`: crea un `TfidfVectorizer` y un modelo de `LogisticRegression`. Define el diccionario `training_data` con sinónimos para cada intención.
- `_train()`: recorre el diccionario, construye listas de frases y etiquetas, aplica `fit_transform` para obtener la matriz TF-IDF y entrena el modelo con `fit`.
- `predict(text)`: recibe un texto, lo limpia, lo transforma con el mismo vectorizador y obtiene la etiqueta más probable.

4.4 Bloque 3: Clase `Robot` con obstáculos (líneas 155–235)

Atributos: `x`, `y`, `direction`, `trail`, `obstacles`.

Inicialización:

```
1 self.obstacles = obstaculos()
2 start_x, start_y = GRID_SIZE // 2, GRID_SIZE // 2
3 if (start_x, start_y) in self.obstacles:
4     # buscar primera celda libre
5     ...
```

Verifica que la posición inicial no es un obstaculo para en caso tal mover la nave a otro cuadrado.

Movimiento (`_step(forward)`):

```
1 dx, dy = DIR_DELTA[self.direction]
2 if not forward: dx, dy = -dx, -dy
3 nx, ny = self.x + dx, self.y + dy
4 if 0 <= nx < GRID_SIZE and 0 <= ny < GRID_SIZE and (nx, ny) not in
   self.obstacles:
5     # actualizar trail y posición
6     return True
7 return False
```

Condición para que la nave no atravesase un obstaculo, si la celda de destino ocupada el código devuelve False haciendo que la nave no pueda ocupar una celda no valida.

4.5 Bloque 4: Enrutador de intenciones `execute_ia_intent` (líneas 238–276)

Recibe una etiqueta (ej. "adelante") y llama al método correspondiente del robot. Devuelve un mensaje que indica si la acción fue exitosa o si fue bloqueada por un obstáculo.

```
1 if intent == "adelante":
2     ok = robot.move_forward()
3     return "Cort: Adelante" + (" if ok else " (Bloqueado por
   obstaculo)")
```

4.6 Bloque 5: Clase `VoiceController` (líneas 279–321)

Controlador de voz que se ejecuta en un hilo independiente. El método `listen_loop`:

- Abre el micrófono y ajusta el umbral de ruido.
- Entra en un bucle infinito escuchando audio con `recognizer.listen`.
- Cuando recibe audio, llama a `recognize_google` (idioma español) para obtener el texto.
- Añade el texto a la lista `self.commands`.

4.7 Bloque 6: Funciones gráficas

4.7.1 `draw_robot(surf, robot)` (líneas 324–375)

Dibuja la estela (círculos naranjas), luego la nave espacial. Los vértices de la nave se definen en coordenadas relativas y se rotan usando trigonometría:

```
1 rx = cx + px * cos(a_rad) - py * sin(a_rad)
2 ry = cy - (px * sin(a_rad) + py * cos(a_rad))
```

Se dibujan sombra, llama del motor, chasis y cabina.

4.7.2 `draw_obstacles(surf, robot)` (líneas 410–423)

Recorre `robot.obstacles` y para cada obstáculo dibuja:

- Un rectángulo de color `C_OBSTACLE`.
- Líneas internas para simular una rejilla.
- Un borde exterior más oscuro.

4.7.3 `draw_panel(...)` (líneas 378–407)

Dibuja el panel inferior con: LED del micrófono, estado de voz, última acción, información de la IA, coordenadas y dirección, texto de ayuda.

4.8 Bloque 7: Bucle principal y código que corre todas las clases y definiciones `main()` (líneas 426–506)

1. **Inicialización:** Pygame, fuentes, robot, controlador de voz, IA. Se lanza el hilo de voz.
2. **Caché de la cuadrícula:** se pre-renderiza en `grid_surf` para optimizar.
3. **Bucle infinito:**
 - Procesa eventos de teclado (flechas).
 - Si hay comandos de voz, extrae el texto. Aplica un **parche para comandos cortos**: si el texto es exactamente "izquierda", "izq", "derecha" o "der", se usa esa palabra como intención (sin pasar por la IA). Caso contrario, llama a `brain.predict`. Esto para evitar un problema de que el sistema se confunda girando de una vez a la izquierda y avanzando.
 - Dibuja: cuadrícula, obstáculos, robot, panel.
 - Actualiza pantalla.

5 Manual de usuario

5.1 Requisitos de instalación

- **Sistema operativo:** Windows, Linux o macOS (con micrófono funcional).
- **Python 3.8+(recomendado 3.10 o mas):** instalado desde <https://python.org>.
- **Entorno de desarrollo recomendado:** VS Code (o cualquier editor de texto).
- **No funciona en Google Colab** porque Colab no tiene acceso al micrófono ni a la ventana gráfica de Pygame.

5.2 Instalación de librerías (ejecutar en terminal)

```
1 pip install pygame
2 pip install SpeechRecognition
3 pip install scikit-learn
4 pip install pyaudio    # necesario en Windows/Linux; en macOS puede
                        ser PyAudio
```

Listing 3: Comandos de instalación

Nota: en Linux puede requerir `sudo apt-get install portaudio19-dev` antes de instalar PyAudio.

5.3 Ejecución en VS Code

1. Abrir la carpeta que contiene `Robot_obs.py`.
2. Abrir una terminal integrada.
3. Ejecutar: `python Robot_obs.py`.
4. Permitir el acceso al micrófono cuando el sistema operativo lo solicite.

5.4 Controles y comandos

5.4.1 Comandos de voz (en español)

Intención	Ejemplos de frases
Adelante	"adelante", "avanza", "camina", "da un paso", "sigue derecho"
Atrás	"atrás", "retrocede", "reversa", "un paso atrás"
Girar izquierda	"izquierda", "gira a la izquierda"
Girar derecha	"derecha", "gira a la derecha"
Girar izquierda y avanzar	"muévete a la izquierda", "gira a la izquierda y avanza"
Girar derecha y avanzar	"muévete a la derecha", "gira a la derecha y avanza"

Si el usuario intenta mover el robot a un obstáculo, el movimiento es cancelado.

5.4.2 Controles de teclado

- **Flecha arriba:** avanzar.
- **Flecha abajo:** retroceder.
- **Flecha izquierda:** girar a la izquierda (sin avanzar).

- **Flecha derecha:** girar a la derecha (sin avanzar).
- **ESC:** salir del programa.

5.5 Interfaz gráfica

- **Cuadrícula:** 20x20 celdas.
- **Obstáculos:** bloques marrones con rejilla.
- **Robot:** nave espacial con estela de propulsión.
- **Panel inferior:**
 - LED verde/rojo indica estado del micrófono.
 - Texto con el último comando ejecutado.
 - Información de la Cortana (intención detectada y porcentaje de confianza).
 - Coordenadas actuales y dirección (Norte, Este, Sur, Oeste).
 - Línea de ayuda con sinónimos.

6 ¿Por qué no funciona en Google Colab?

Google Colab es un entorno Jupyter notebook en la nube que:

- No proporciona acceso directo al hardware de audio (micrófono).
- No tiene servidor X11 para mostrar ventanas de Pygame (aunque se puede usar `display` con `pygame.display.init()`, es complejo y poco fiable).
- La librería `speech_recognition` requiere un micrófono físico o virtual.

Por tanto, el programa está diseñado para ejecutarse **localmente** en el ordenador del usuario.

7 Posibles errores y soluciones

- **Error:** No module named 'pygame' → instalar con `pip install pygame`.
- **El micrófono no funciona** → verificar permisos del sistema y que el dispositivo esté activo.

8 Limitaciones del sistema

A pesar de los buenos resultados, el sistema presenta varias limitaciones que deben ser conocidas por el usuario y que abren líneas de trabajo futuro.

- **Reconocimiento limitado a comandos entrenados:** El clasificador solo puede detectar las intenciones que fueron incluidas en el conjunto de entrenamiento (**adelante**, **atras**, etc.). Una frase como *“Salta”* o *“Detente”* no será reconocida tampoco *“Arriba”*.
- **Dataset pequeño:** El diccionario de entrenamiento contiene menos de 100 frases. Esto es suficiente para un prototipo, pero insuficiente para un sistema robusto que maneje variaciones del lenguaje real, para entrenar estos sistemas se requieren muchos elementos, pues pueden haber palabras que hagan al robot entrar en conflicto con similitudes a 2 comandos.
- **No hay aprendizaje en tiempo real:** El modelo se entrena una sola vez al inicio. No aprende de los errores ni de nuevas frases.
- **Dependencia de Internet:** El reconocimiento de voz requiere conexión a los servidores de Google. Sin Internet, el robot no puede recibir comandos de voz (aunque el teclado sigue funcionando).
- **No planifica rutas:** No existe navegación autónoma.
- **Sensibilidad al ruido ambiente:** EL microfono puede detectar otros elementos cuando hay mucho ruido y no escuchar al emisor.
- **Latencia:** El tiempo entre que el usuario habla y el robot se mueve puede ser de 2 a 4 segundos debido a la comunicación con Google y el procesamiento.

4

9 Conclusión

El programa `Robot_obs.py` integra exitosamente reconocimiento de voz, clasificación de intenciones mediante aprendizaje automático, gráficos interactivos y detección de obstáculos. Es una herramienta educativa que demuestra conceptos de inteligencia artificial, programación concurrente y desarrollo de interfaces. La inclusión de obstáculos añade realismo y desafío al control del robot, además de el desafío al usuario a aguantarse que el robot no lo escuche y sea rebelde(toca regañarlo).